

Université de l'Unité Africaine (UUA)

Dr Hervé W. OUEDRAOGO

Algèbre relationnel: ¹

IG2

Année Universitaire 2025-2026

¹Ce cours peut être librement copié et distribué. Toute remarque, correction ou suggestion doit être adressée à l'auteur: herveoueder@yahoo.fr

Objectif général-Objectifs spécifiques

L'objectif général de ce cours est de donner aux apprenants les connaissances de base de l'algèbre relationnel. Plus spécifiquement:

- i) de maîtriser les concepts de base de l'algèbre relationnel;
- ii) de comprendre la notion de schéma d'une base de donnée;
- iii) de connaître les différentes formes normales d'un schéma;
- iv) de maîtriser le langage de requête;
- v) être capable de faire les différentes opérations sur des tables.

Contents

Sommaire	i
1 Les concepts de base	1
1.1 Les concepts de bases	2
1.2 Les dépendances fonctionnelles	7
1.3 Les formes normales	10
2 Algèbre relationnel	16
2.1 Les opérateurs de base	17
2.2 Les opérateurs additionels	23
2.3 Le langage algébrique	27

Les concepts de base

section Définitions L'ALGÈBRE RELATIONNELLE est un outil de requête utilisé pour le traitement d'information en entrée et donne d'autres en sortie. Il utilise plusieurs opérations pour effectuer cette action. Les opérations de requête d'algèbre relationnelle sont effectuées de manière répétitive sur une relation.

Le modèle relationnel a été introduit dans les années 1970 par Edgar Frank Codd, alors informaticien chez IBM, comme moyen de définir et représenter les données de grandes bases de données d'une manière simple mais rigoureuse.

Il formalise des opérations applicables à des relations afin d'en extraire des informations pour répondre à des questions. Ces opérations deviendront le socle théorique des langages d'accès aux bases de données comme SQL.

En résumé, c'est un langage de manipulation de données dont la fondamentale est la table relationnelle.

1.1 Les concepts de bases

1.1.1 Ensemble

Un ensemble est une collection d'objets (éléments) rassemblés d'après une ou plusieurs propriétés communes. Ces propriétés sont suffisantes pour affirmer qu'un objet appartient ou pas à un ensemble. Un ensemble peut être constitué d'objets, de choses, de personnes, de nombres, de chiffres etc.

Exemple 1.1.1

- *l'ensemble des étudiants de l'université de l'Unité Africaine;*
- *l'ensemble des nombre pairs;*
- *l'ensemble des article de la boutique LIZA MALL.*

1.1.2 Le domaine

C'est un ensemble de valeur ayant le même champs sémantique et caractérisé par un nom. C'est l'ensemble des valeurs possibles dans lequel on peut puiser des données. La notion de domaine est assimilable à celle de type de données utilisée en algorithmique ou programmation :

Exemple 1.1.2

- *jeux vidéo {Word of Warcraft, Starcraft, Vampire, Civilization }*
- *Sexe {F, M}*
- *Ville du Burkina, {Ouagadougou, Kaya, Bobo, Banfora,}*

Remarque 1.1.1 *Deux domaines peuvent avoir les même valeurs bien que sémantiquement différents.*

Exemple 1.1.3

- *Numéro de téléphone {0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }*
- *Année de naissance {0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }*

Un domaine peut être plus précis :

Exemple 1.1.4

- *jours du mois* : $[1;31]$;
- *jours de la semaine* : $\{\text{dimanche, lundi, mardi, ..., vendredi, samedi}\}$;
- *mois de l'année* : $\{\text{janvier, février, mars, ..., novembre, décembre}\}$;
- *lettres minuscules de l'alphabet* : $[a;z]$

1.1.3 Produit cartésien

Le produit cartésien de deux ensembles E et F est l'ensemble de tous les couples, dont la première composante appartient à E et la seconde à F , on le note $E \times F$.

soit $D_1, D_2, \dots, D_n$, n domaines, une relation R est contenu dans le produit cartésien $D_1 \times D_2 \times \dots \times D_n$.

1.1.4 Relation

Une relation est un sous ensemble d'un produit cartésien de plusieurs domaines, elle peut aussi être vue comme une table de valeurs (ensemble de valeurs uniques et non ordonnées).

Exemple 1.1.5 Soit les domaines :

- *joursmois* : $[1;31]$;
- *moisannee* : $\{\text{janvier, février, mars, ..., novembre, décembre}\}$;

Une relation *Anniversaire*(*jourMois*, *moisAnnee*), contient la liste des dates pour lesquelles un anniversaire est à souhaiter et est contenu dans le produit cartésien de l'ensemble des éléments de *jourssemaine* par l'ensemble des éléments *joursmois* et de *moisannee*.

le produit cartésien sera composé de toutes les combinaisons possibles de ces 2 ensembles mais la relation *Anniversaire* ne retiendra qu'une partie de ce dernier. n même domaine peut être utilisé plusieurs fois dans une relation.

$$\text{Anniversaire} \subseteq \text{joursmois} \times \text{moisAnnee}$$

Une relation est également un prédicat à N variables : par exemple "l'anniversaire a lieu le jourmois du mois de moisannee" soit $\text{Anniversaire}(\text{jourMois}, \text{moisAnnee})$.

Une proposition devient "l'anniversaire a lieu le 16 du mois de mai".

Un tuple est une proposition VRAIE : exemple : $\text{Anniversaire}(16, \text{"mai"})$.

1.1.5 Attribut

Un attribut est l'une des composants d'une relation. Chaque attribut est identifié et est associé à un domaine de valeur. U Le nom que l'on donne à un attribut défini généralement l'attribut et à un sens, mais doit toujours être différent du nom du domaine.

Par exemple, pour la relation *Anniversaire*, on peut définir les attributs suivants :

- jour appartenant au domaine jourmois;
- mois appartenant au domaine moisannee.

Un attribut doit avoir un nom unique dans une relation.

Seul le fait d'appartenir au même domaine permet aux attributs d'être comparés (et donc, des attributs qui ne sont pas du même domaine ne peuvent être comparés sémantiquement, même si leur type syntaxique est le même.

Par exemple, un numéro de mois et une année sont tous deux des nombres entiers naturels, mais ils ne sont pas comparables car ils n'ont pas le même domaine, et donc pas le même sens (sémantique différente). autrement dit, plusieurs attributs peuvent avoir le même domaine.

1.1.6 Schéma d'une relation

Le schéma de relation définit l'ensemble des attributs et domaines qui interviennent dans la relation. le schéma d'une relation se définit par le nom de la relation ainsi que ses attributs, on le note : $R(A_1, A_2, \dots, A_n)$, avec R le nom de la relation, $A_1 \in D_1, A_2 \in D_2, \dots, A_n \in D_n$

Exemple 1.1.6

- *Anniversaire (jour, mois);*

- *Etudiant (nom, prenom, code).*

Définition 1.1.1

Le degré ou arité d'une relation correspond à son nombre d'attributs.

Le degré de la relation Anniversaire est 2 et 3 pour la relation Etudiant.

Définition 1.1.2

Un-nuplet (en anglais : tuple) est une collection ordonnée de n éléments. Il ne peut y avoir 2 n -nuplets identiques dans une relation (pas de doublons).

Exemple 1.1.7

- Des n -nuplets de la relation Anniversaire pourraient être : (16, mai), (29, juin), (6, octobre), etc.
- la relation Etudiant pourraient être : (KANTAGBA, Valentin, 808167), (ZANRE, Hawa, 808397), (SAVADOGO, Wassilatou, 808603), etc.

Définition 1.1.3

La cardinalité de la relation correspond à son nombre de n -nuplets.

Une manière courante de représenter les relations est la forme tabulaire (sous forme de tableau) comme ci-dessous :

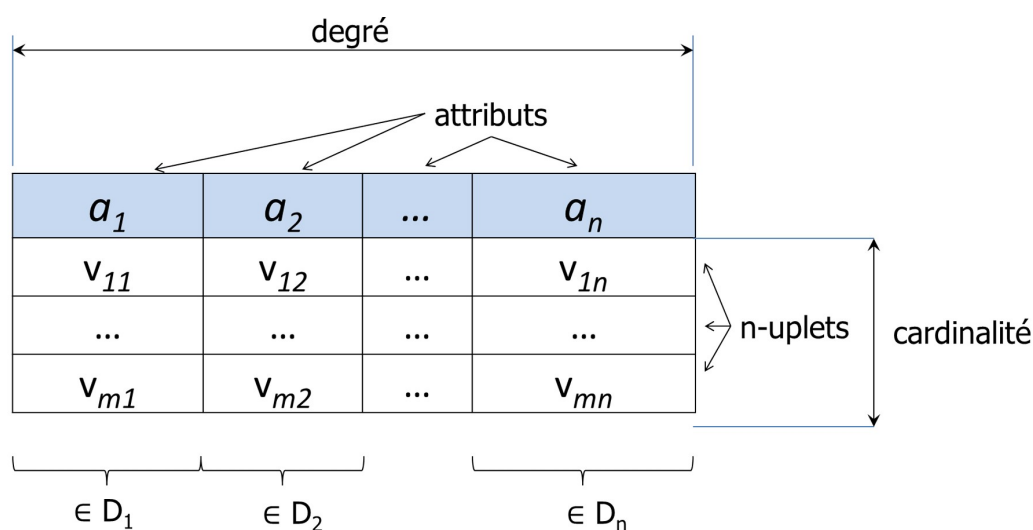


Figure 1.1:

- l'entête représente la définition de la relation;
- le corps représente la valeur de la relation (appelée relval).

Exemple 1.1.8 Soit le domaine lettre correspondant aux lettres de $[a; z]$.

Soit la relation R définie par les attributs A, B et C :

- en compréhension (ou en intention) : $R(A, B, C)$; Degré de la relation : 3
- en extension : $R = ((a,b,c), (d,a,f), (c,b,d))$; Cardinalité de la relation 3
- sous forme tabulaire :

R	A	B	C
	a	b	c
	d	a	f
	c	b	d

Figure 1.2:

Exemple 1.1.9 On considère la base de données de l'UUA relatif aux nouveau étudiants inscrit et est composée des relations "Etudiant", "Formation" et "Inscription" relier par les tables suivantes :

- $Etudiant(numero, nom, prenom, dateNaissance)$

Etudiant	numero	nom	prénom	dateNaissance
	1	KONSEIGA	Nouria	15/01/1990
	9	BAMA	Rolande	20/07/1999
	23	DIALLO	Moctar	03/12/1994
	34	SIMPORE	Faouzia	15/12/1994
	105	KANAZOE	Djibril	20/10/1995

- Formation(niveau, filière)

Formation	Niveau	Filière
	BTS1	IG
	BTS2	FC
	Licence	BQ
	Master	COM

- Inscription (niveau, numéro)

Inscription	niveau	numéro
	BTS1	1
	BTS1	9
	BTS1	34
	Licence	1
	Licence	23
	Licence	34
	Master	1

1.1.7 Schéma d'une base de donnée

C'est l'ensemble des schémas des relations qui la compose. ce schéma indique comment les informations sont organisées et est représenté la plupart du temps par plusieurs tables.

1.2 Les dépendances fonctionnelles

Une dépendance fonctionnelle est une contrainte entre deux ensembles d'attributs dans une relation (table). Dans une relation R , on dit qu'il y a dépendance fonctionnelle entre un ensemble d'attributs A et un ensemble d'attributs B , ou que l'ensemble A d'attributs détermine l'ensemble B d'attributs (et on écrit $A \twoheadrightarrow B$) si quand deux n -uplets coïncident sur leurs attributs A , alors ils coïncident aussi sur leurs attributs B . autrement dit, toutes valeurs de l'attribut B est identifiée de manière unique par les valeurs de l'attribut A .

Une dépendance fonctionnelle est désignée par une flèche " $\hookrightarrow$ ". La dépendance fonctionnelle de A sur B est représentée par $A \hookrightarrow B$.

quelques règles importantes pour la dépendance fonctionnelle :

- règle réflexive : Si X est un ensemble d'attributs et $Y \subset X$, alors X contient une valeur de Y;
- règle réflexive : Si X est un ensemble d'attributs et $Y \subset X$, alors X contient une valeur de Y;
- règle d'augmentation : lorsque $X \hookrightarrow Y$ est valable et que Z est un ensemble d'attributs, alors $XZ \hookrightarrow YZ$ est également valable. Cela consiste à ajouter des attributs qui ne modifient pas les dépendances de base;
- règle de transitivité : Cette règle est très similaire à la règle transitive en algèbre si $X \hookrightarrow Y$ est valable et $Y \hookrightarrow Z$ est valable, alors $X \hookrightarrow Z$ est également valable.

1.2.1 Notion de clef dans une relation

La notion de clef permet de maintenir l'intégrité des valeurs dans les relations : c'est un attribut ou en ensemble d'attributs de la relation et dont tout les autres attributs dépendent fonctionnellement. autrement dit c'est un ou plusieurs attributs qui détermine tout les autres.

Soit $R(A_1, A_2, \dots, A_n)$ et $C \subseteq \{A_1, A_2, \dots, A_n\}$.

C est une clé de la relation si et seulement si $C \hookrightarrow A_1, A_2, \dots, A_n$.

1.2.1.1 La clef candidate

Attribut, ou ensemble minimal d'attributs, qui permet l'identification d'un n-uplet parmi tous les n-uplets d'une relation (un numéro, un code INSEE, une adresse mail). On la représente généralement soulignée en pointillés. La clef candidate permet d'indexé chaque ligne d'une table et de manière différente.

1.2.1.2 La clef primaire

Attribut, ou ensemble d'attributs, choisi parmi les clefs candidates, qui permet l'identification d'un n-uplet parmi tous les n-uplets d'une relation; on la représente généralement soulignée. Autrement dit, c'est une clef choisie parmi les clés candidates, Ou encore attribut permettant d'identifier chaque ligne d'une table d'une manière unique.

1.2.1.3 La clef étrangère

Attribut (ou groupe d'attributs) clef étrangère dans une relation est une clef primaire dans une autre relation; elle matérialise un lien logique entre ces 2 relation; on la représente généralement préfixée, ou suffixée, du caractère #. La clef étrangère identifie une colonne ou un ensemble d colonnes d'une autre table, autrement dit attribut d'une table (table A) qui est clé primaire d'une table (table B)

1.2.2 Les règles d'intégrités

Les contraintes d'intégrité sont des règles qui édictent les propriétés que doivent respecter les champs de la base de données (les champs correspondent aux attributs du modèle relationnel) afin de garantir leur cohérence, leur pertinence et leur validité.

Dans une base de données, une contrainte d'intégrité permet de garantir la cohérence des données lors des mises à jour de la base. En effet, les données d'une base ne sont pas indépendantes, mais obéissant à des règles sémantiques, après chaque mise à jour, le système contrôle qu'aucune contrainte d'intégrité n'est violée.

Le modèle relationnel impose les contraintes structurelles suivantes :

- intégrité du domaine;
- intégrité de la clef;
- intégrité référentielle.

Les contraintes sont définies au moment de la création des tables.

1.2.2.1 Intégrité du domaine

Pour assurer l'intégrité d'un domaine dans une base de donnée, les valeurs d'une colonne dans une table doivent appartenir exclusivement à ce domaine.

1.2.2.2 Intégrité de la clef

Les valeurs de la clef primaire doivent être unique et non null (non encore initialisé).

1.2.2.3 Intégrité référentielle

Les valeurs des clefs étrangères doivent être null ou les valeur de la clef primaire auxquelles elle font référence.

1.3 Les formes normales

1.3.1 définition

La théorie de la normalisation met en évidence les données indésirables dans une relation, elle définit les critères des relations désirables appelées formes normales tel que:

- les redondances;
- les valeurs NULL;
- le boucles infinies.

Elle définit un processus consistant à décomposer une relation non normalisée en un ensemble équivalent de relations normalisées. Cette décomposition doit se faire suivant :

- décomposer les relations du schéma relationnel sans perte d'informations;
- obtenir des relations canoniques ou de base du monde réel et qui s'imbrique.

Le schéma de départ est le schéma universel de la base. Par raffinement successifs on obtient des sous relations sans perte d'informations et qui ne seront pas affectées lors des mises à jour (non redondance).

Les formes normales (ou database normalization en anglais) sont un ensemble de bonnes pratiques permettant de concevoir des modèles de bases de données sans redondance et en même temps robustes, elles permettent de décomposer et d'organiser logiquement les données à informatiser afin de manipuler des données atomiques (c'est à dire qu'on ne peut plus subdiviser) et des relations entre les différentes données. Concrètement, les formes normales sont un ensemble de règles qui permettent de créer des bases de données avec des tables contenant uniquement les colonnes nécessaires et des relations correctement modélisées.

1.3.2 Vérification et normalisation du modèle relationnel

On dit qu'un modèle relationnel est vérifié s'il répond aux conditions suivantes :

- tous les attributs sont présents dans une entité ou table;
- aucun attribut n'est redondant;
- toutes les tables ont une clé primaire;
- tous les attributs sont élémentaires (non décomposable ou se traitent comme un tout, exemple l'attribut adresse peut-être décomposable).

Les règles de normalisation du modèle conceptuel permettent d'assurer :

- la non redondance des données;
- l'intégrité des données;
- la facilité de mise à jour.

La normalisation des données repose sur un ensemble de règles appelées formes normales. Les règles sont caractérisées par des niveaux, chaque règle s'appuyant sur la précédente. Ainsi donc vous ne pouvez appliquer un deuxième niveau que si le premier est respecté, un troisième si le deuxième est respecté, et ainsi de suite. Six formes normales sont disponibles, mais une base de données est considérée comme normalisée lorsqu'elle atteint le troisième niveau. Voyons ce qu'il en est pour chacune d'entre elles.

1.3.3 La première forme normale ou la 1FN

cette étape élimine les entrées redondantes dans un groupe, chaque enregistrement doit être unique. Cela signifie qu'il faut une clé primaire, pas de n valeurs dans une liste, pas de groupes répétitifs et des colonnes atomiques où chaque cellule contient un enregistrement unique et distinct qui ne peut pas être divisé davantage.

Autrement dit, les tables ne doivent pas contenir des groupes de données répétitives. Si tel est le cas, il faut sortir le groupe de données et créer une autre entité qui va contenir ce groupe de données.

Exemple 1.3.1

ELEVE (no_elv, nom, prenom, liste_notes)

Un attribut ne peut pas être un ensemble de valeurs

Décomposition

ELEVE (no_elv, nom, prenom)

NOTE (no_elv, no_matiere, note)

1.3.4 La deuxième forme normale ou la 2FN

Après avoir satisfait aux règles de la première forme normale, vous pouvez maintenant passer à la deuxième forme normale. L'objectif est de supprimer les entrées répétitives en veillant à ce que les sous-groupes de données figurant dans plusieurs lignes de tableaux soient extraits et présentés dans un nouveau tableau avec des connexions qui les traversent. Pour plus de clarté, tous les sous-ensembles de vos données existant sur plusieurs lignes sont placés dans des tableaux distincts.

Ensuite, vous pouvez créer des relations entre les nouvelles tables et les clés d'étiquetage. Cela implique de supprimer les dépendances partielles lorsque des tables relationnelles ayant des clés primaires avec deux attributs ou plus sont mappées vers une nouvelle table avec des clés d'étiquettes correspondant à la clé primaire. Le lien peut être établi à l'aide de contraintes de clés étrangères.

La deuxième forme normale ne s'applique qu'aux tables ayant une clé primaire composée.

Une Table ou une relation est en deuxième forme normale 2FN, si elle est :

- en 1FN;
- tous les attributs de la relation dépendent toute de la clé (notion de clé primaire composée) et non d'une partie de la clé.

C'est la phase d'identification des clés, cette étape évite certaines redondances, tout attribut doit dépendre fonctionnellement de la totalité de la clé.

Dans une relation, les attributs sont divisés en deux groupes, le premier groupe correspond aux attributs de la clé et le second groupe les attributs non clé. La 2FN exige donc que les attributs du deuxième groupe ne doivent pas dépendre d'une partie de la clé.

Exemple 1.3.2

La relation suivante est en 1FN mais pas en 2FN.

COMMANDE (*date*, *no_cli*, *no_pro*, *qte*, *prixUHT*)

elle n'est pas en 2FN car la clé = (date, no_cli, no_pro), et le prixUHT ne dépendent que de no_pro.

Décomposition

COMMANDE (*date*, *no_cli*, *no_pro*, *qte*)

PRODUIT (*no_pro*, *prixUHT*)

1.3.5 Troisième forme normale ou 3NF

Pour réaliser la 3NF, la deuxième forme normale doit être respectée, ce qui, à son tour, exige que la première forme normale soit ratifiée. La règle est que les modèles de données 3NF ne doivent dépendre que de la clé primaire – pas de dépendances fonctionnelles transitives. Si vous modifiez la clé primaire (en la supprimant, en l'insérant ou en la remplaçant), toutes les données qui dépendent de la clé primaire sont acheminées vers une nouvelle table.

Imaginez un enregistrement contenant des noms, des adresses et des sexes. Si vous changez le nom d'une personne, son sexe peut être modifié. Pour remédier à ce problème, le genre 3NF reçoit une clé étrangère et une nouvelle table pour

stocker les données.

Une table ou une relation est en troisième forme normale (3FN) si :

- elle est déjà en 2FN;
- tous les attributs non clé dépendent uniquement de la clé. Il n'y a pas de dépendance entre deux attributs non clé.

Exemple 1.3.3

La relation suivante est en 2FN mais pas en 3FN.

*VOITURE (**matricule**, marque, modèle, puissance)*

on vérifie facilement qu'elle est en 2FN ;

*elle n'est pas en 3FN car la clé = **matricule**, et la puissance dépend de (marque, modèle).*

Décomposition

*VOITURE (**matricule**, marque, modèle)*

*MODELE (marque, **modèle**, puissance)*

1.3.6 Quatrième forme normale (4NF)

Les variantes supérieures de la 3NF – la forme normale de Boyce-Codd (BCNF) – attribuées à Raymond F. Boyce et Edgar F. Codd, s'appuient sur le cadre de Boyce-Codd, qui prend en compte les dépendances multivaluées. Maintenant que les données sont normalisées, elles ne sont plus couramment utilisées. Toutefois, les entreprises qui travaillent avec des ensembles de données complexes qui changent fréquemment devraient envisager de satisfaire aux normes restantes.

Comme prévu, la 4NF ne peut être réalisée que si la 3NF a été respectée. Toutes les dépendances non triviales sont éliminées, à l'exception des niveaux pour une clé candidate.

Une relation (table) est en BCNF si :

- elle est déjà en 3FN;
- Les seules dépendances fonctionnelles sont celles dans lesquelles une clé (entière si composée) détermine un attribut.

Lorsque la clé primaire est composée, il arrive que certaine partie de la clé détermine un attribut non clé. La table peut être en 2FN, en 3FN mais pas en BCNF.

1.3.7 Cinquième et sixième forme normale (5NF et 6NF)

Également appelée forme normale projet-joint (PJ/NF), cette étape élimine les dépendances cycliques dans les tables et les attributs. Elle vise les cas où une combinaison d'attributs est utilisée comme clés dans une table. Une table entre dans cette catégorie si elle est conforme à la 4NF et ne peut être divisée en tables plus petites sans utiliser de données. L'objectif est d'identifier et d'isoler les relations sémantiques multiples.

Avant d'arriver à la 6NF, le niveau le plus élevé, nous avons la forme normale de la clé de domaine (DK/NF). Une base de données ne peut pas avoir de contraintes au-delà des options de clé et de domaine. Dans ce cas, toutes les relations de contrainte sont une séquence logique de clés et de domaines définis. Avec la 6NF, une base de données doit respecter les règles de la 5NF et ne prend pas en charge les dépendances de jointure non triviales.

ALGÈBRE RELATIONNEL

On appelle algèbre relationnelle un ensemble d'opérations simples sur des tables relationnelles, à partir desquelles des opérations plus complexes sont définies par composition. Ils définissent donc un petit langage de manipulation de données.

L'élément fondamental du modèle de base de données relationnelle est la table relationnelle. Une table relationnelle est une représentation en extension d'une relation définie sur un certain domaine.

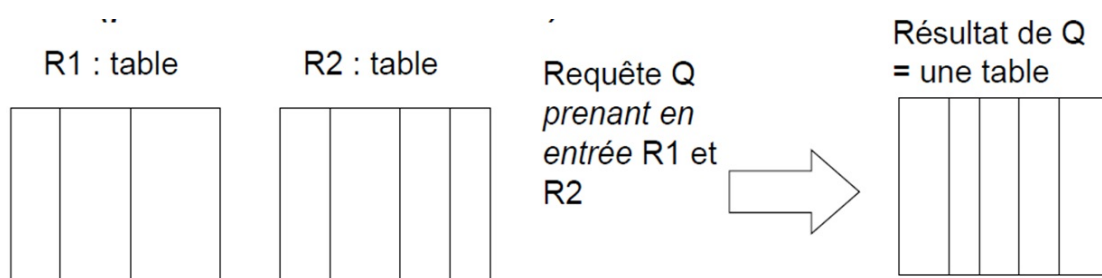
La représentation d'information sous forme relationnelle est intéressante car les fondements mathématiques du relationnel, outre qu'ils permettent une modélisation logique simple et puissante, fournissent également un ensemble de concepts pour manipuler formellement l'information ainsi modélisée.

Ainsi une algèbre relationnelle, sous forme d'un ensemble d'opérations formelles, permet d'exprimer des questions, ou requêtes, posées à une représentation relationnelle, sous forme d'expressions algébriques.

Ces requêtes algébriques sont alors appelés plans d'exécution. Autant les tuples retournés sont les mêmes pour tous ces plans d'exécution, autant le temps nécessaire pour obtenir ces tuples, lui peut être très différent.

Une requête s'applique à des relations (tables) pour en créer une nouvelle.

Exemple 2.0.4



Remarque 2.0.1

- *La table resultat d'une requête met toujours en jeu les mêmes attributs quelque en soit la nature de la requête.*
- *Une requête s'exprime grâce à des noms d'attributs et de tables.*
- *Aucunne connaissance du contenu des table ne doit être nécessaire pour bien comprendre une requête.*

L'algèbre relationnelle est principalement composée par les cinq opérateurs de base et les trois opérateurs additionnels suivants :

1. Opérateurs de base :

- union;
- différence;
- projection;
- restriction;
- produit cartésien.

2. Opérateurs additionels :

- intersection;
- jointure;
- division.

2.1 Les opérateurs de base

Pour illustrer les différents opérateurs nous considérons les schémas de relations suivants $R(\underline{A}, B, C, \#D)$, $S(\underline{G}, H, I, J)$, $U(\underline{D}, E, F)$, $V(\underline{K}, \underline{L}, M)$ et $W(\underline{M})$ où les clés primaires sont soulignées et la clé étrangère est préfixée par un #.

A	B	C	D
1	2	3	4
2	3	1	4
3	4	1	2
4	3	1	2

E	F	G	H
1	2	3	4
2	4	1	3
3	4	1	2
4	2	1	3

D	I	J
2	3	4
3	1	2
4	2	3

K	L	M
1	2	1
1	2	2
2	1	1
2	1	2
2	1	3
3	2	1

M
1
2
3

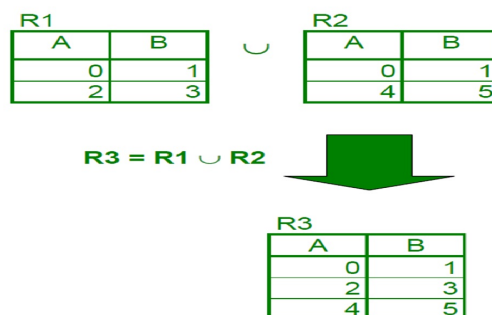
2.1.1 L'union, $\cup$

L'expression $R \cup S$ crée une relation comprenant tous les tuples existant dans l'une ou l'autre des relations R et S. Il existe une condition impérative : les deux relations doivent avoir le même schéma, c'est-à-dire même nombre d'attributs, mêmes noms et mêmes types.

$R \cup S$ est la table contenant toutes les lignes de A et toutes les lignes de B sans doublon.

A	B	C	D
1	2	3	4
2	3	1	4
3	4	1	2
4	3	1	2
1	2	3	4
2	4	1	3
3	4	1	2
4	2	1	3

Exemple 2.1.1



2.1.2 La différence, $-$

Comme l'union, la différence s'applique à deux relations qui ont le même schéma. L'expression $R - S$ a alors pour résultat tous les tuples de R qui ne sont pas dans S . $R - S$ est la table contenant toutes les lignes de R qui ne se trouvent pas dans S . La différence est le seul opérateur qui permet d'exprimer des requêtes comportant une négation (on veut rejeter quelque chose, on ne veut pas des lignes ayant telle propriété).

$$R - S$$

A	B	C	D
2	3	1	4
4	3	1	2

Exemple 2.1.2

R1			R2	
A	B		A	B
0	1	-	0	1
2	3		4	5

R3 = R1 - R2



R3	
A	B
2	3

2.1.3 La projection, π

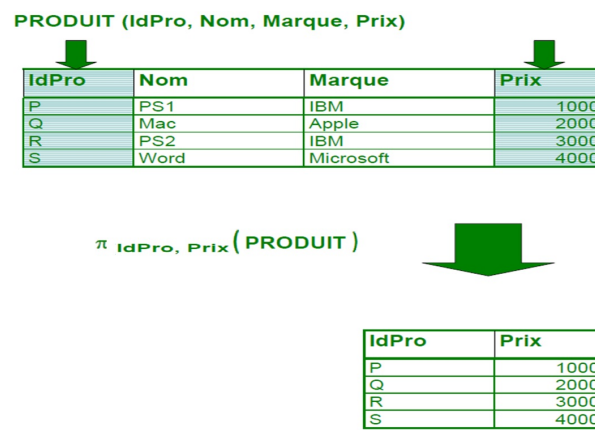
La projection est une opération unaire (c'est à dire portant sur une seule relation). La projection de $R1$ sur une partie de ses attributs $A1, A2, \dots$ produit une relation $R2$ dont le schéma est restreint aux attributs mentionnés en opérande, comportant les mêmes tuples que $R1$, et dont les doublons sont éliminés.

Projeter sur un ensemble de colonnes d'une table T , revient à supprimer de la table celles qui ne sont pas dans l'ensemble puis à retourner l'ensemble des tuples composés uniquement des attributs présents en paramètre de la projection. On note : $\pi_{listeAttributs}(R)$.

$$\Pi_{B,C}(R)$$

B	C
2	3
3	1
4	1

Exemple 2.1.3 *Quels sont les références et les prix des produits ?*



2.1.4 La restriction ou Sélection, σ

La restriction est une opération unaire (c'est à dire portant sur une seule relation). La restriction de R1, étant donnée une condition C, produit une relation R2 de même schéma que R1 et dont les tuples sont les tuples de R1 vérifiant la condition C.

On appelle condition une assertion valant vrai ou faux sur une ligne de table. La restriction d'une relation R1 est une relation R2 de même schéma n'ayant que les n-uplets de R1 répondant à la condition énoncée.

On notera : $R2 = \sigma_{\text{condition}}(R1)$.

la restriction d'une relation R1 suivant le critère où condition est une relation d'égalité ou d'inégalité entre 2 attributs ou entre un attribut et une valeur. La restriction permet d'extraire les n-uplets qui satisfont une condition. Elle correspond à un découpage horizontal.

La selection sur la condition notée σ_{cond} correspond à l'algorithme suivant :

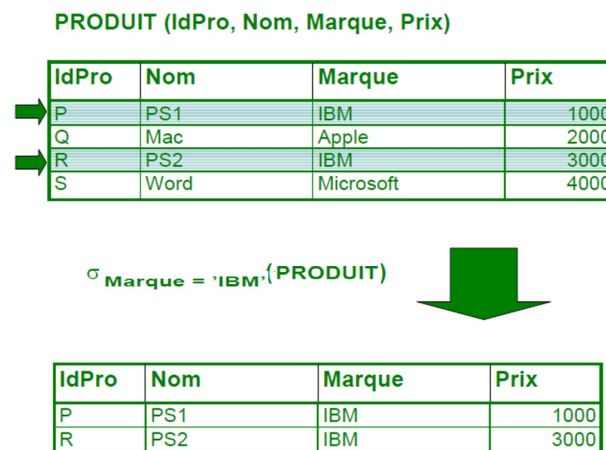
```

pour chaque ligne de T faire
  si cond (ligne) = vrai
    alors garder la ligne
  sinon ne pas la garder
fin pour

```

La sélection sur une condition consiste donc à garder les lignes de la table vérifiant la condition, elle a pour effet de supprimer des lignes, mais chaque ligne garde l'ensemble de ses attributs.

Exemple 2.1.4 Quelles sont les produits de marque IBM ?



2.1.5 Le produit cartésien, $\times$

Le premier opérateur binaire, et le plus important, est le produit cartésien. Le produit cartésien entre deux relations R et S se note $R \times S$, et permet de créer une nouvelle relation où chaque tuple de R est associé à chaque tuple de S . Autrement dit, pour chaque ligne de R fabriquer autant de lignes qu'il y a de lignes dans S par concaténation.

Exemple 2.1.5

$$U \times W$$

D	I	J	M
2	3	4	1
2	3	4	2
2	3	4	3
3	1	2	1
3	1	2	2
3	1	2	3
4	2	3	1
4	2	3	2
4	2	3	3

Exemple 2.1.6

Voici deux relations R et S :

A	B
a	b
x	y

C	D
c	d
u	v
x	y

Et voici le résultat de $R \times S$:

A	B	C	D
a	b	c	d
a	b	u	v
a	b	x	y
x	y	c	d
x	y	u	v
x	y	x	y

En lui-même, le produit cartésien ne présente pas un grand intérêt puisqu'il associe aveuglément chaque ligne de R à chaque ligne de S . Il ne prend vraiment son sens qu'associé à l'opération de sélection, ce qui permet d'exprimer des jointures, opération fondamentale qui sera détaillée plus loin.

2.2 Les opérateurs additionnels

2.2.1 L'intersection, $\cap$

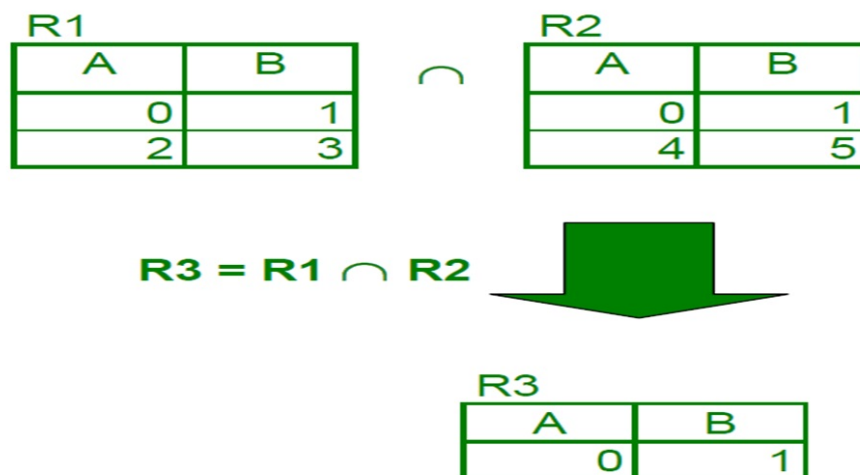
L'intersection entre deux relations R_1 et R_2 de même schéma est une relation R_3 de schéma identique ayant pour n -uplets les n -uplets communs à R_1 et R_2 . On notera : $R_3 = R_1 \cap R_2$. L'intersection contient toutes les lignes communes aux deux tables. Notons que l'intersection n'est pas une opération de base, car elle est équivalente à deux opérations de différence successives.

Exemple 2.2.1

$R \cap S$

A	B	C	D
1	2	3	4
3	4	1	2

Exemple 2.2.2



2.2.2 La jointure,

La jointure de deux relations $R1$ et $R2$ est une relation $R3$ dont les n-uplets sont obtenus en concaténant les nuplets de $R1$ avec ceux de $R2$ et en ne gardant que ceux qui vérifient la condition de liaison. On notera : $R3 = R1 \times R2(\text{condition})$, la jointure de $R1$ avec $R2$ suit le schéma suivant :

- le schéma de la relation résultat de la jointure est la concaténation des schémas des opérandes (s'il y a des attributs de même nom, il faut les renommer);
- Les n-uplets de $R1 \times R2(\text{condition})$ sont tous les couples $(u1, u2)$ d'un n-uplet de $R1$ avec un n-uplet de $R2$ qui satisfont condition;
- la jointure de deux relations $R1$ et $R2$ est le produit cartésien des deux relations suivi d'une restriction;
- La condition de liaison doit être du type : $\{\text{attribut1}\} \odot \{\text{attribut2}\}$, où : attribut1 appartient à la 1ère relation et attribut2 à la 2ème relation et $\odot$ est un opérateur de comparaison (égalité ou inégalité).

La jointure permet de composer 2 relations à l'aide d'un critère de liaison, c'est l'opération permettant de coller au bout des lignes de la table à toutes les lignes de la table B vérifiant la condition de jointure on la note : $\bowtie_{\text{Cond}}$.

$T_1 \bowtie_{cond} T_2 = \sigma_{cond}(T_1 \times T_2)$ Est le résultat de l'algorithme suivant :

```

pour chaque ligne de  $T_1$  faire
  pour chaque ligne de  $T_2$  faire
    lig = concaténer ligne  $T_1$ 
    et ligne  $T_2$ 
    si  $cond(lig) = \text{vrai}$  garder lig
  fin pour
fin pour

```

Exemple 2.2.3

R1(A, B, C)

A	B	C
A1	B1	10
A2	B2	10
A3	B3	20
A4	B4	30

×

R2(U, V)

U	V
10	V1
20	V2
30	V3

$R1 \times R2 (R1.C = R2.U)$



A	B	C	U	V
A1	B1	10	10	V1
A1	B2	10	10	V1
A3	B3	20	20	V2
A4	B4	30	30	V3

Exemple 2.2.4

VENTE As V

IdCli	IdPro	Date	Qte
X	P	1/1/98	1
Y	Q	2/1/98	1
Z	P	3/1/98	1

×

PRODUIT As P

IdPro	Désignation	Prix
P	PS	100
Q	Mac	100

$VENTE \times PRODUIT (V.IdPro=P.IdPro)$



Idcli	IdPro	Date	Qte	Désignation	Prix
X	P	1/1/98	1	PS	100
Y	Q	2/1/98	1	Mac	100
Z	P	3/1/98	1	PS	100

2.2.3 La division, $\div$

La division est une opération binaire (c'est à dire portant sur deux relations). La division de R par S a pour résultat T tel que T comporte le plus grand ensemble possible de tuples qui concaténés à ceux de S donnent toujours un tuple de R.

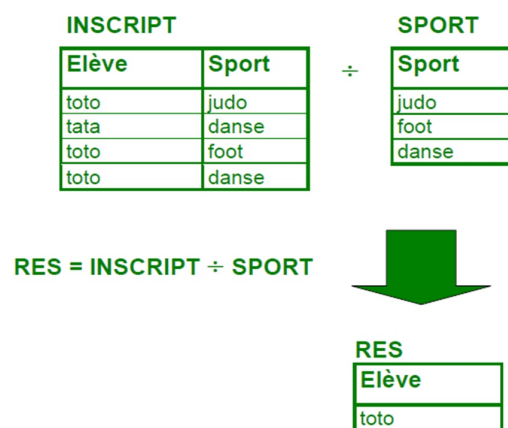
Soit deux relations R1 ($A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m$) et R2 ($B_1, B_2, \dots, B_m$). Si le schéma de R2 est un sous-schéma de R1, La division de R1 par R2 est une relation R3 dont :

- le schéma est le sous-schéma complémentaire de R2 par rapport à R1;
- un n-uplet ($a_1, a_2, \dots, a_n$) appartient à R3 si ($a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_m$) appartient à R1 pour tous ($b_1, b_2, \dots, b_m$) appartenant à R2.

On notera : $R3 = R1 \div R2$ (la division de R1 par R2).

la division permet de rechercher dans une relation les sous n-uplets qui sont complétés par tous ceux d'une autre relation. Elle permet de répondre à des questions qui sont formulées avec le quantificateur universel pour tout, quelque soit

Exemple 2.2.5 *Quels sont les élèves qui sont inscrits à tous les sports ?*



2.3 Le langage algébrique

L'algèbre relationnelle fait partie des fondements des bases de données relationnelles, utilisé en interne par le SGBD pour évaluer les requêtes. Le langage SQL quant à lui est destiné aux utilisateurs du SGBD et, à l'heure actuelle, est le langage standard d'interrogation. Il marie d'une certaine façon l'algèbre relationnelle et le calcul relationnel de n-uplets.

Le langage algébrique permet de formuler une question par une suite d'opérations de l'algèbre relationnelle.

Exemple 2.3.1 *Requêtes sur le schéma CLIENT, PRODUIT, VENTE*

1. CLIENT (IdCli, nom, ville)
2. PRODUIT (IdPro, désignation, marque, prix)
3. VENTE (IdCli, IdPro, date, qte)

2.3.1 Requête 1 : Donner les no des produits de marque Apple et de prix < 25000F

$$R1 = \sigma_{\text{marque}=\text{Apple}}(\text{PRODUIT})$$

$$R2 = \sigma_{\text{prix}<25000}(\text{PRODUIT})$$

$$R3 = R1 \cap R2$$

$$\text{RESUL} = \pi_{\text{IdPro}}(R3)$$

2.3.2 Requête 2 : Donner les no des clients ayant acheté un produit de marque Apple

$$R1 = \sigma_{\text{marque}=\text{Apple}}(\text{PRODUIT})$$

$$R2 = R1 \times \text{VENTE}(R1.\text{IdPro} = \text{VENTE}.\text{IdPro})$$

$$\text{RESUL} = \pi_{\text{IdCli}}(R2)$$

2.3.3 Requête 3 : Donner les no des clients n'ayant acheté que des produits de marque Apple

$$R1 = VENTE \times PRODUIT(VENTE.IdPro = PRODUIT.IdPro)$$

$$R2 = \sigma_{marque=Apple}(R1)$$

$$R3 = \pi_{IdCli}(R2)$$

$$R4 = \sigma_{marque \neq Apple}(R1)$$

$$R5 = \pi_{IdCli}(R4)$$

$$RESUL = R3 - R5$$

2.3.4 Requête 4 : Donner les no des clients ayant acheté tous les produits de marque Apple

$$R1 = \sigma_{marque=Apple}(PRODUIT)$$

$$R2 = \pi_{IdPro}(R1)$$

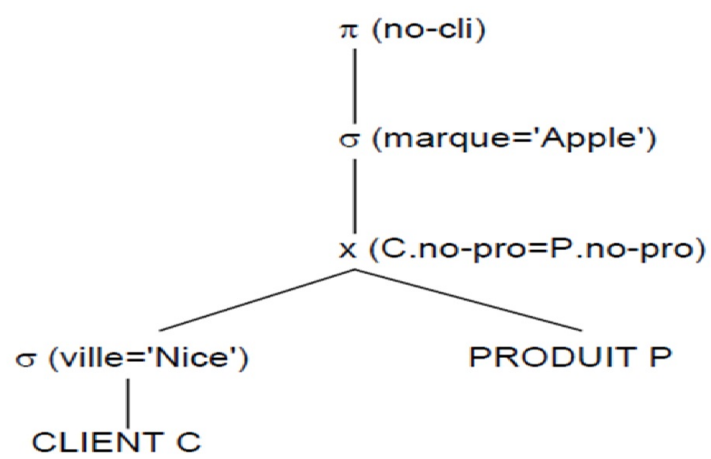
$$R3 = \pi_{IdCli, IdPro}(VENTE)$$

$$RESUL = R3 \div R2$$

Une question peut être représentée par un arbre.

2.3.5 Requête 5 : Quels sont les clients de Nice ayant acheté un produit de marque Apple?

Plusieurs arbres équivalents peuvent être déduits d'un arbre donné à l'aide de règles de transformation simples, telles que permutation des jointures et restrictions, permutation des projections et des jointures, etc. Ces transformations sont à la base des techniques d'optimisation de requêtes.



References

- [1] M. BOUZEGHOUB, M. JOUVE, P. PUCHERAL : LE MODELE RELATIONNEL. ALGEBRE, LANGAGES, APPLICATIONS. HERMES, 1998
- [2] C. CHRISMENT : BASES DE DONNEES RELATIONNELLES: CONCEPTS, MISE EN OEUVRE ET EXERCICES, HERMES, 2008
- [3] CLOUSE : ALGEBRE RELATIONNELLE, ENI SERVICES, 2008
- [4] Elisabetta De Maria : Algèbre relationnelle, Université de Nice Sophia-Antipolis, 2021